

Personalized Learning Path

Architecture MVP Fusion AI

Workflows, orchestrateurs, inputs/outputs, tables Supabase et nœuds

Architecture relationnelle sans JSON/JSONB pour les inserts Supabase

Hackathon Fusion AI Academy - Juillet 2026

Table des matières

1. 1. Périmètre et décisions d'architecture
2. 2. Architecture globale du MVP
3. 3. Contrat commun entre workflows
4. 4. ORCH-01 - Résultat et analyse
5. 5. ORCH-02 - Génération du plan
6. 6. ORCH-03 - Décision et distribution
7. 7. ORCH-04 - Suivi et dashboard
8. 8. Tables Supabase minimales
9. 9. Dashboard MVP
10. 10. Ordre d'implémentation et tests

1. Périmètre et décisions d'architecture

Ce document décrit un premier MVP répondant au BPMN TO-BE du use case de 1 à 9 : réception du résultat, calcul des scores, détection des lacunes, diagnostic pédagogique, génération du plan, validation humaine, archivage, envoi à l'étudiant et notification de l'enseignant.

- Un seul étudiant est traité par exécution.
- Une matière et une évaluation par exécution.
- Trois à cinq compétences par résultat.
- Les calculs de score et de lacunes sont déterministes.
- L'IA est utilisée uniquement pour le diagnostic, la génération et la régénération du plan.
- Aucun plan n'est envoyé avant validation de l'enseignant.
- Les tables n'utilisent aucune colonne JSON ou JSONB obligatoire.
- Les tableaux skills et activities circulent entre workflows, puis sont insérés ligne par ligne avec un nœud Loop.

2. Architecture globale du MVP

ORCH-01 - Résultat et analyse

WF-01 Import et validation

WF-02 Stockage du résultat

WF-03 Scoring et détection des lacunes

WF-04 Diagnostic pédagogique IA

ORCH-02 - Génération du plan

WF-05 Sélection des ressources

WF-06 Génération du plan IA

WF-07 Validation technique du plan

ORCH-03 - Décision et distribution

WF-08 Décision enseignant

WF-09 Régénération du plan

WF-10 Notification étudiant et enseignant

ORCH-04 - Suivi et dashboard

WF-11 Mise à jour de progression

WF-12 Dashboard API

Orchestrateur	Objectif	Workflows
ORCH-01	Transformer le résultat corrigé en analyse pédagogique	WF-01 à WF-04
ORCH-02	Sélectionner les ressources, générer puis contrôler le plan	WF-05 à WF-07
ORCH-03	Enregistrer la décision humaine et distribuer le plan	WF-08 à WF-10
ORCH-04	Mettre à jour la progression et alimenter le dashboard	WF-11 à WF-12

3. Contrat commun entre workflows

Entrée commune

```
{
  "transaction_id": "TX-20260715-001",
  "correlation_id": "CORR-20260715-001",
  "source_workflow": "ORCH-01",
  "student_code": "STD001",
  "assessment_code": "ASSESS-PYTHON-001",
  "teacher_code": "T001"
}
```

Sortie succès commune

```
{
  "success": true,
  "transaction_id": "TX-20260715-001",
  "correlation_id": "CORR-20260715-001",
  "workflow_id": "WF-01",
  "status": "completed",
  "next_action": null,
  "data": {},
  "errors": []
}
```

Sortie erreur commune

```
{
  "success": false,
  "transaction_id": "TX-20260715-001",
  "correlation_id": "CORR-20260715-001",
  "workflow_id": "WF-01",
  "status": "failed",
  "error_code": "VALIDATION_ERROR",
  "error_message": "Les données reçues sont invalides.",
  "retryable": false,
  "data": null,
  "errors": [
    {
      "field": "skills",
      "reason": "minimum_3_skills_required"
    }
  ]
}
```

4. ORCH-01 - Résultat et analyse

ORCH-01 - Résultat et analyse

Rôle : Orchestrer validation, stockage, scoring, détection des lacunes et diagnostic pour un seul étudiant.

Endpoint : POST /orchestrator/student-analysis

Tables utilisées : students, assessments, student_results, student_skill_results

Schéma du workflow

Webhook Trigger -> Initialize Context -> WF-01 -> If valid? -> WF-02 -> WF-03 -> If gaps? -> WF-04 -> Build Response -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir le résultat corrigé.
2	Function initialize_context	Créer transaction_id, correlation_id et contrôler les champs de base.
3	HTTP Request WF-01	Normaliser et valider les données.
4	If-Else validation_success	Arrêter avec 422 si invalide.
5	HTTP Request WF-02	Stocker étudiant, évaluation, résultat et compétences.
6	If-Else storage_success	Arrêter avec 500 si stockage incomplet.
7	HTTP Request WF-03	Calculer score, lacunes et priorité.
8	If-Else gaps_detected	Choisir suivi standard ou diagnostic IA.
9	HTTP Request WF-04	Générer le diagnostic pédagogique.
10	Function build_response	Construire la réponse finale.
11	Respond to Webhook	Retourner 200, 422 ou 500.

Règles principales

- Ne pas lancer WF-04 en absence de lacune significative.
- Ne jamais conserver le webhook ouvert pour une décision humaine.

Input

```
{
  "transaction_id": "TX-001",
  "correlation_id": "CORR-001",
  "student_code": "STD001",
  "student_name": "Yassine El Amrani",
  "student_email": "yassine@example.com",
  "class_code": "L1-INFO-A",
  "teacher_code": "T001",
  "assessment_code": "ASSESS-PYTHON-001",
  "assessment_title": "Évaluation Python - Fondamentaux",
  "subject": "Python",
  "idempotency_key": "ASSESS-PYTHON-001-STD001-V1",
  "skills": [
    {
      "skill_code": "PY-VARIABLES",
      "skill_name": "Variables",
      "score": 80,
      "max_score": 100
    },
    {
      "skill_code": "PY-LOOPS",
      "skill_name": "Boucles",
      "score": 30,
      "max_score": 100
    },
    {
      "skill_code": "PY-FUNCTIONS",
      "skill_name": "Fonctions",
      "score": 40,
      "max_score": 100
    }
  ]
}
```

```

    }
  ]
}

```

Output succès

```

{
  "success": true,
  "transaction_id": "TX-001",
  "correlation_id": "CORR-001",
  "workflow_id": "ORCH-01",
  "status": "analysis_completed",
  "next_action": "generate_learning_plan",
  "data": {
    "student_id": 1,
    "assessment_id": 1,
    "student_result_id": 101,
    "global_score": 55,
    "gaps_detected": true,
    "priority_level": "high",
    "diagnosis_summary": "Bonne maîtrise des variables, difficulté critique sur les boucles.",
    "skills_count": 5
  },
  "errors": []
}

```

Output erreur type

```

{
  "success": false,
  "workflow_id": "ORCH-01",
  "status": "validation_failed",
  "error_code": "INVALID_STUDENT_RESULT",
  "retryable": false,
  "data": null,
  "errors": [
    {
      "field": "skills",
      "reason": "minimum_3_skills_required"
    }
  ]
}

```

WF-01 - Import et validation

Rôle : Fusionner normalisation et validation du résultat étudiant.

Endpoint : POST /student-result/validate

Schéma du workflow

Webhook -> Extract Input -> Normalize Identifiers -> Normalize Student -> Normalize Skills -> Validate Required Fields -> Validate Skill Count -> Validate Scores -> Detect Duplicates -> Aggregate Errors -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir le payload.
2	Function extract_input	Extraire les champs et protéger contre les valeurs absentes.
3	Function normalize_identifiers	Trim, uppercase des codes.
4	Function normalize_student	Nettoyer nom et email.
5	Function normalize_skills	Convertir score/max_score en nombres.
6	Function validate_required	Vérifier les champs obligatoires.
7	Function validate_skill_count	Imposer 3 à 5 compétences.
8	Function validate_scores	Vérifier $0 \leq \text{score} \leq \text{max_score}$ et $\text{max_score} > 0$.
9	Function detect_duplicates	Détecter skill_code dupliqué.
10	Function aggregate_errors	Construire errors[].
11	If-Else is_valid	Choisir succès ou erreur.

Ordre	Nœud	Fonction
12	Respond to Webhook	Retourner le payload normalisé.

Règles principales

- Aucun insert Supabase dans ce workflow.
- Les skills restent un tableau uniquement pour le transport entre workflows.

Input

```
{
  "transaction_id": "TX-001",
  "correlation_id": "CORR-001",
  "student_code": " std001 ",
  "student_name": " Yassine El Amrani ",
  "student_email": " YASSINE@EXAMPLE.COM ",
  "class_code": " l1-info-a ",
  "teacher_code": " t001 ",
  "assessment_code": " assess-python-001 ",
  "assessment_title": " Évaluation Python - Fondamentaux ",
  "subject": " python ",
  "idempotency_key": "ASSESS-PYTHON-001-STD001-V1",
  "skills": [
    {
      "skill_code": " py-loops ",
      "skill_name": " Boucles ",
      "score": "30",
      "max_score": "100"
    },
    {
      "skill_code": " py-functions ",
      "skill_name": " Fonctions ",
      "score": "40",
      "max_score": "100"
    },
    {
      "skill_code": " py-lists ",
      "skill_name": " Listes ",
      "score": "70",
      "max_score": "100"
    }
  ]
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-001",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-01",
  "status": "validated",
  "next_action": "store_student_result",
  "data": {
    "valid": true,
    "student_code": "STD001",
    "student_name": "Yassine El Amrani",
    "student_email": "yassine@example.com",
    "class_code": "L1-INFO-A",
    "teacher_code": "T001",
    "assessment_code": "ASSESS-PYTHON-001",
    "assessment_title": "Évaluation Python - Fondamentaux",
    "subject": "Python",
    "idempotency_key": "ASSESS-PYTHON-001-STD001-V1",
    "skills_count": 3,
    "skills": [
      {
        "skill_code": "PY-LOOPS",
        "skill_name": "Boucles",

```

```

    "score": 30,
    "max_score": 100
  }
],
},
"errors": []
}

```

Output erreur type

```

{
  "success": false,
  "workflow_id": "WF-01",
  "status": "validation_failed",
  "error_code": "INVALID_STUDENT_RESULT",
  "retryable": false,
  "data": {
    "valid": false,
    "skills_count": 2
  },
  "errors": [
    {
      "field": "skills",
      "reason": "minimum_3_skills_required"
    },
    {
      "field": "skills[1].score",
      "reason": "score_out_of_range"
    }
  ]
}

```

WF-02 - Stockage du résultat

Rôle : Enregistrer l'étudiant, l'évaluation, le résultat et une ligne par compétence.

Endpoint : POST /student-result/store

Tables utilisées : students, assessments, student_results, student_skill_results

Schéma du workflow

Webhook -> Prepare Student -> Upsert Student -> Prepare Assessment -> Upsert Assessment -> Upsert Student Result -> Prepare Skill Rows -> Loop Skills -> Insert Skill Row -> Count Stored Rows -> Verify Storage -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir le payload validé.
2	Function prepare_student	Construire les colonnes simples de students.
3	Supabase Upsert students	Upsert sur student_code.
4	Function prepare_assessment	Construire les colonnes simples de assessments.
5	Supabase Upsert assessments	Upsert sur assessment_code.
6	Function prepare_result	Construire student_results avec idempotency_key.
7	Supabase Upsert student_results	Upsert sur idempotency_key.
8	Function prepare_skill_rows	Ajouter student_result_id à chaque skill.
9	Loop	Itérer sur chaque compétence.
10	Supabase Insert student_skill_results	Insérer une ligne simple.
11	Function count_inserted	Compter les lignes réellement insérées.
12	Function verify_storage	Comparer received, prepared et stored.
13	Respond to Webhook	Retourner stored ou storage_incomplete.

Règles principales

- Aucun tableau n'est inséré dans Supabase.
- skills_received = skills_prepared = skills_stored obligatoire.
- Un replay idempotent ne doit pas créer de doublons.

Input

```
{
  "transaction_id": "TX-001",
  "correlation_id": "CORR-001",
  "idempotency_key": "ASSESS-PYTHON-001-STD001-V1",
  "student": {
    "student_code": "STD001",
    "full_name": "Yassine El Amrani",
    "email": "yassine@example.com",
    "class_code": "L1-INFO-A",
    "teacher_code": "T001"
  },
  "assessment": {
    "assessment_code": "ASSESS-PYTHON-001",
    "title": "Évaluation Python - Fondamentaux",
    "subject": "Python",
    "class_code": "L1-INFO-A",
    "teacher_code": "T001"
  },
  "skills": [
    {
      "skill_code": "PY-VARIABLES",
      "skill_name": "Variables",
      "score": 80,
      "max_score": 100
    },
    {
      "skill_code": "PY-LOOPS",
      "skill_name": "Boucles",
      "score": 30,
      "max_score": 100
    }
  ]
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-001",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-02",
  "status": "stored",
  "next_action": "analyze_scores",
  "data": {
    "student_id": 1,
    "assessment_id": 1,
    "student_result_id": 101,
    "skills_received": 5,
    "skills_prepared": 5,
    "skills_stored": 5,
    "idempotency_key": "ASSESS-PYTHON-001-STD001-V1",
    "idempotent_replay": false
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "WF-02",
  "status": "storage_incomplete",
  "error_code": "SKILL_STORAGE_MISMATCH",
  "retryable": true,
  "data": {
    "student_result_id": 101,
    "skills_received": 5,
    "skills_prepared": 5,

```

```

    "skills_stored": 4
  },
  "errors": [
    {
      "field": "skills",
      "reason": "skill_storage_mismatch"
    }
  ]
}

```

WF-03 - Scoring et détection des lacunes

Rôle : Calculer le score global, classer les compétences et déterminer la priorité.

Endpoint : POST /student/analyze-scores

Tables utilisées : student_results, student_skill_results

Schéma du workflow

Webhook -> Validate IDs -> Select Skill Rows -> Calculate Percentages -> Calculate Global Score -> Classify Skills -> Calculate Counts -> Determine Priority -> Loop Skill Updates -> Update Student Result -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir student_result_id.
2	Function validate_identifiers	Vérifier les identifiants.
3	Supabase Select student_skill_results	Charger les compétences.
4	Function calculate_percentages	Calculer score / max_score * 100.
5	Function calculate_global_score	Calculer la moyenne pondérée.
6	Function classify_skills	mastered, fragile, gap, critical_gap.
7	Function calculate_counts	Compter chaque niveau.
8	Function determine_priority	Déterminer standard, low, medium ou high.
9	Loop	Parcourir les lignes compétence.
10	Supabase Update student_skill_results	Mettre à jour pourcentage et gap_level.
11	Supabase Update student_results	Mettre à jour global_score, priority_level, analysis_status.
12	Respond to Webhook	Retourner l'analyse complète.

Règles principales

- 70-100 mastered; 50-69 fragile; 35-49 gap; 0-34 critical_gap.
- Aucun appel IA.

Input

```

{
  "transaction_id": "TX-002",
  "correlation_id": "CORR-001",
  "student_result_id": 101,
  "student_code": "STD001",
  "assessment_code": "ASSESS-PYTHON-001"
}

```

Output succès

```

{
  "success": true,
  "transaction_id": "TX-002",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-03",
  "status": "score_analysis_completed",
  "next_action": "generate_pedagogical_diagnosis",
  "data": {
    "student_result_id": 101,
    "global_score": 55,
    "gaps_detected": true,
    "priority_level": "high",
    "mastered_count": 2,

```

```

    "fragile_count": 1,
    "gap_count": 1,
    "critical_gap_count": 1,
    "skills": [
      {
        "skill_result_id": 1003,
        "skill_code": "PY-LOOPS",
        "skill_name": "Boucles",
        "score": 30,
        "max_score": 100,
        "percentage": 30,
        "gap_level": "critical_gap"
      }
    ]
  },
  "errors": []
}

```

Output erreur type

```

{
  "success": false,
  "workflow_id": "WF-03",
  "status": "scoring_failed",
  "error_code": "NO_SKILL_ROWS",
  "retryable": false,
  "data": {
    "student_result_id": 101
  },
  "errors": [
    {
      "field": "student_skill_results",
      "reason": "no_rows_found"
    }
  ]
}

```

WF-04 - Diagnostic pédagogique IA

Rôle : Expliquer les forces, faiblesses et impacts sans modifier les scores.

Endpoint : POST /ai/pedagogical-diagnosis

Tables utilisées : student_results, student_skill_results

Schéma du workflow

Webhook -> Load Student Context -> Prepare Agent Context -> AI Agent -> Parse JSON -> Validate Fields and Skills -> Flatten Text Fields -> Update Student Result -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir student_result_id.
2	Supabase Select context	Charger étudiant, score et compétences.
3	Function prepare_agent_context	Préparer un contexte limité et fiable.
4	Agent Pedagogical Diagnosis	Appeler le LLM.
5	Function parse_llm_json	Parser la réponse.
6	Function validate_diagnosis	Vérifier champs obligatoires et compétences existantes.
7	Function flatten_text_fields	Convertir forces/faiblesses en texte séparé par virgules.
8	Supabase Update student_results	Stocker les champs TEXT du diagnostic.
9	Respond to Webhook	Retourner le diagnostic.

Règles principales

- L'IA ne modifie aucun score ni gap_level.
- strengths et weaknesses sont stockés en TEXT, pas en JSON.
- Retour attendu strictement structuré.

Input

```
{
  "transaction_id": "TX-002",
  "correlation_id": "CORR-001",
  "student_result_id": 101,
  "student_name": "Yassine El Amrani",
  "subject": "Python",
  "global_score": 55,
  "priority_level": "high",
  "skills": [
    {
      "skill_code": "PY-VARIABLES",
      "skill_name": "Variables",
      "percentage": 80,
      "gap_level": "mastered"
    },
    {
      "skill_code": "PY-LOOPS",
      "skill_name": "Boucles",
      "percentage": 30,
      "gap_level": "critical_gap"
    }
  ]
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-002",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-04",
  "status": "diagnosis_generated",
  "next_action": "generate_learning_plan",
  "data": {
    "student_result_id": 101,
    "diagnosis_summary": "L'étudiant maîtrise les variables mais présente une difficulté critique sur les boucles.",
    "diagnosis_strengths": "Variables, Listes",
    "diagnosis_weaknesses": "Boucles, Fonctions",
    "pedagogical_impact": "Difficulté à automatiser des traitements.",
    "priority_justification": "Une intervention rapide est nécessaire.",
    "confidence": "high",
    "model_name": "gemini-flash-lite"
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "WF-04",
  "status": "invalid_ai_output",
  "error_code": "INVALID_AI_RESPONSE",
  "retryable": true,
  "data": null,
  "errors": [
    {
      "field": "summary",
      "reason": "missing_required_field"
    }
  ]
}
```

5. ORCH-02 - Génération du plan

ORCH-02 - Génération du plan

Rôle : Orchestrer sélection des ressources, génération IA et validation technique.

Endpoint : POST /orchestrator/generate-plan

Tables utilisées : student_results, student_skill_results, learning_plans, learning_plan_activities

Schéma du workflow

Webhook -> Initialize Plan Context -> WF-05 -> If Resources Found? -> WF-06 -> If Generated? -> WF-07 -> If Valid? -> Build Response -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir student_result_id et contraintes.
2	Function initialize_plan_context	Normaliser durée et limites.
3	HTTP Request WF-05	Sélectionner les ressources.
4	If-Else resources_found	Retourner 404 si aucune ressource.
5	HTTP Request WF-06	Générer et stocker le plan.
6	If-Else generation_success	Retourner 500 en cas d'échec.
7	HTTP Request WF-07	Contrôler le plan.
8	If-Else output_valid	Retourner 422 ou pending_teacher_validation.
9	Function build_response	Construire la réponse.
10	Respond to Webhook	Retourner le résultat.

Règles principales

- Durée cible 5 à 7 jours.
- Le plan doit rester pending_teacher_validation.

Input

```
{
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "student_result_id": 101,
  "student_code": "STD001",
  "preferred_duration_days": 7,
  "max_daily_minutes": 60,
  "max_resources_per_skill": 2
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "workflow_id": "ORCH-02",
  "status": "pending_teacher_validation",
  "next_action": "teacher_plan_decision",
  "data": {
    "learning_plan_id": 201,
    "plan_code": "PLAN-STD001-PYTHON",
    "version": 1,
    "duration_days": 7,
    "activities_count": 4,
    "validation_score": 100
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "ORCH-02",
  "status": "quality_validation_failed",
  "error_code": "INVALID_LEARNING_PLAN",
  "retryable": true,
  "next_action": "regenerate_plan",
  "data": {
    "learning_plan_id": 201,
    "validation_score": 60
  },
  "errors": []
}
```

WF-05 - Sélection des ressources

Rôle : Associer des ressources existantes aux compétences gap et critical_gap.

Endpoint : POST /resources/select

Tables utilisées : student_skill_results

Schéma du workflow

Webhook -> Select Gap Skills -> Load Resource Configuration -> Filter by Skill -> Filter Active and Difficulty -> Rank -> Limit per Skill -> Merge -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir student_result_id.
2	Supabase Select student_skill_results	Charger gap et critical_gap.
3	Function load_resource_catalog	Charger un catalogue statique ou une source simple.
4	Function filter_by_skill	Garder les ressources de la bonne compétence.
5	Function filter_availability	Garder les ressources actives.
6	Function rank_resources	Prioriser difficulté et durée.
7	Function limit_resources	Maximum N ressources par compétence.
8	Function merge_results	Créer la liste finale.
9	Respond to Webhook	Retourner les ressources.

Règles principales

- Pour minimiser les tables, le catalogue peut être configuré dans le workflow ou dans une source externe simple.
- Aucune insertion Supabase.

Input

```
{
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "student_result_id": 101,
  "max_resources_per_skill": 2
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-05",
  "status": "resources_selected",
  "next_action": "generate_ai_plan",
  "data": {
    "resources_found": true,
    "resources_count": 3,
    "resources": [
```

```

{
  "resource_code": "RES-LOOPS-01",
  "skill_code": "PY-LOOPS",
  "title": "Introduction aux boucles for",
  "resource_type": "video",
  "resource_url": "https://example.com/loops-video",
  "estimated_minutes": 15
},
{
  "resource_code": "RES-FUNCTIONS-01",
  "skill_code": "PY-FUNCTIONS",
  "title": "Créer une fonction Python",
  "resource_type": "exercice",
  "resource_url": "https://example.com/functions",
  "estimated_minutes": 30
}
]
},
"errors": []
}

```

Output erreur type

```

{
  "success": false,
  "workflow_id": "WF-05",
  "status": "no_resources_found",
  "error_code": "RESOURCE_NOT_FOUND",
  "retryable": false,
  "data": {
    "resources_found": false,
    "resources_count": 0
  },
  "errors": [
    {
      "field": "skill_code",
      "reason": "no_resource_for_gap"
    }
  ]
}

```

WF-06 - Génération du plan IA

Rôle : Générer un plan structuré et insérer une ligne par activité.

Endpoint : POST /ai/learning-plan/generate

Tables utilisées : learning_plans, learning_plan_activities

Schéma du workflow

Webhook -> Load Diagnosis -> Prepare Plan Context -> AI Agent -> Parse JSON -> Validate Basic Shape -> Insert Learning Plan -> Prepare Activity Rows -> Loop Activities -> Insert Activity -> Verify Counts -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir contexte et ressources.
2	Supabase Select student_results	Charger diagnostic et priorité.
3	Function prepare_plan_context	Limiter les champs transmis au LLM.
4	Agent Learning Plan	Générer le plan.
5	Function parse_plan_json	Parser la réponse.
6	Function validate_basic_shape	Contrôler titre, durée, activités.
7	Supabase Insert learning_plans	Insérer une ligne de plan avec champs simples.
8	Function prepare_activity_rows	Ajouter learning_plan_id.
9	Loop	Parcourir activités[].
10	Supabase Insert learning_plan_activities	Insérer une activité par ligne.
11	Function verify_activity_storage	Comparer received/prepared/stored.
12	Respond to Webhook	Retourner le plan généré.

Règles principales

- Aucune colonne JSON/JSONB.
- objective_summary et success_criteria_summary sont des champs TEXT.
- activities_received = activities_prepared = activities_stored.

Input

```
{
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "student_result_id": 101,
  "student_name": "Yassine El Amrani",
  "subject": "Python",
  "diagnosis_summary": "Difficultés critiques sur les boucles et faiblesse sur les fonctions.",
  "diagnosis_weaknesses": "Boucles, Fonctions",
  "priority_level": "high",
  "duration_days": 7,
  "max_daily_minutes": 60,
  "resources": [
    {
      "resource_code": "RES-LOOPS-01",
      "skill_code": "PY-LOOPS",
      "title": "Introduction aux boucles for",
      "resource_type": "video",
      "resource_url": "https://example.com/loops-video",
      "estimated_minutes": 15
    }
  ]
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-06",
  "status": "plan_generated",
  "next_action": "validate_learning_plan",
  "data": {
    "learning_plan_id": 201,
    "plan_code": "PLAN-STD001-PYTHON",
    "version": 1,
    "duration_days": 7,
    "objective_summary": "Renforcer la maîtrise des boucles et des fonctions.",
    "success_criteria_summary": "Atteindre au moins 65 sur les boucles.",
    "activities_received": 2,
    "activities_prepared": 2,
    "activities_stored": 2,
    "plan_status": "generated"
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "WF-06",
  "status": "activity_storage_incomplete",
  "error_code": "ACTIVITY_STORAGE_MISMATCH",
  "retryable": true,
  "data": {
    "activities_received": 2,
    "activities_prepared": 2,
    "activities_stored": 1
  },
  "errors": [
    {
```



```

    "field": "activities",
    "reason": "activity_storage_mismatch"
  }
]
}

```

WF-07 - Validation technique du plan

Rôle : Vérifier la cohérence du plan stocké avant présentation à l'enseignant.

Endpoint : POST /learning-plan/validate

Tables utilisées : student_skill_results, learning_plans, learning_plan_activities

Schéma du workflow

Webhook -> Load Plan -> Load Activities -> Load Gap Skills -> Validate Duration -> Validate Activity Count
-> Validate Skill Alignment -> Validate Resource Fields -> Aggregate Errors -> Update Plan Status -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir learning_plan_id.
2	Supabase Select learning_plans	Charger le plan.
3	Supabase Select learning_plan_activities	Charger les activités.
4	Supabase Select student_skill_results	Charger les lacunes.
5	Function validate_duration	Contrôler durée.
6	Function validate_activity_count	Au moins une activité.
7	Function validate_skill_alignment	Chaque activité cible une lacune.
8	Function validate_resource_fields	Code, URL et titre présents.
9	Function aggregate_errors	Calculer validation_score.
10	If-Else output_valid	Choisir statut.
11	Supabase Update learning_plans	Mettre pending_teacher_validation ou validation_failed.
12	Respond to Webhook	Retourner les contrôles.

Règles principales

- Le workflow valide les lignes déjà stockées.
- Aucune dépendance à une colonne JSON.

Input

```

{
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "learning_plan_id": 201,
  "student_result_id": 101,
  "max_duration_days": 7,
  "max_daily_minutes": 60
}

```

Output succès

```

{
  "success": true,
  "transaction_id": "TX-003",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-07",
  "status": "plan_validation_completed",
  "next_action": "teacher_plan_decision",
  "data": {
    "learning_plan_id": 201,
    "output_valid": true,
    "validation_score": 100,
    "activities_count": 2,
    "invalid_activities_count": 0,
    "plan_status": "pending_teacher_validation"
  },
}

```

```
"errors": []  
}
```

Output erreur type

```
{  
  "success": false,  
  "workflow_id": "WF-07",  
  "status": "plan_validation_failed",  
  "error_code": "INVALID_LEARNING_PLAN",  
  "retryable": true,  
  "next_action": "regenerate_plan",  
  "data": {  
    "learning_plan_id": 201,  
    "output_valid": false,  
    "validation_score": 60,  
    "invalid_activities_count": 1  
  },  
  "errors": [  
    {  
      "field": "activity.skill_code",  
      "reason": "skill_not_in_student_gaps"  
    }  
  ]  
}
```

6. ORCH-03 - Décision et distribution

ORCH-03 - Décision et distribution

Rôle : Orchestrer la décision enseignant, la régénération ou l'envoi.

Endpoint : POST /orchestrator/teacher-decision

Tables utilisées : learning_plans, learning_plan_activities, workflow_events

Schéma du workflow

Webhook -> Normalize Decision -> WF-08 -> Switch Decision: approve -> WF-10; regenerate -> WF-09; reject -> Build Response -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir décision.
2	Function normalize_decision	Normaliser approve/regenerate/reject.
3	HTTP Request WF-08	Enregistrer la décision.
4	Switch decision	Router selon la décision.
5	HTTP Request WF-09	Régénérer si demandé.
6	HTTP Request WF-10	Notifier si approuvé.
7	Function build_response	Construire le résultat final.
8	Respond to Webhook	Retourner immédiatement.

Règles principales

- Aucune attente humaine dans un webhook ouvert.
- Approve autorise l'envoi; regenerate crée une version; reject arrête le processus.

Input

```
{
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "learning_plan_id": 201,
  "teacher_code": "T001",
  "decision": "approve",
  "comment": "Plan adapté au niveau de l'étudiant."
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "workflow_id": "ORCH-03",
  "status": "plan_distributed",
  "next_action": "track_student_progress",
  "data": {
    "learning_plan_id": 201,
    "decision": "approve",
    "plan_status": "sent",
    "student_notification_sent": true,
    "teacher_notification_sent": true
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "ORCH-03",
  "status": "decision_failed",
  "error_code": "INVALID_DECISION",
}
```

```

"retryable": false,
"data": {
  "learning_plan_id": 201
},
"errors": [
  {
    "field": "decision",
    "reason": "unsupported_value"
  }
]
}

```

WF-08 - Décision enseignant

Rôle : Vérifier l'enseignant et mettre à jour le statut du plan.

Endpoint : POST /teacher/plan-decision

Tables utilisées : learning_plans, workflow_events

Schéma du workflow

Webhook -> Normalize Decision -> Load Plan -> Verify Teacher -> Validate Current Status -> Switch Decision
-> Update Plan -> Insert Event -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir la décision.
2	Function normalize_decision	Normaliser les valeurs.
3	Supabase Select learning_plans	Charger le plan.
4	Function verify_teacher	Comparer teacher_code.
5	Function validate_current_status	Exiger pending_teacher_validation.
6	Switch decision	Approve, regenerate ou reject.
7	Supabase Update learning_plans	Mettre decision, comment et status.
8	Supabase Insert workflow_events	Tracer la décision.
9	Respond to Webhook	Retourner la prochaine action.

Règles principales

- Décisions autorisées : approve, regenerate, reject.
- teacher_comment est un champ TEXT.

Input

```

{
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "learning_plan_id": 201,
  "teacher_code": "T001",
  "decision": "approve",
  "comment": "Plan validé."
}

```

Output succès

```

{
  "success": true,
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-08",
  "status": "teacher_decision_recorded",
  "next_action": "send_plan_notification",
  "data": {
    "learning_plan_id": 201,
    "teacher_code": "T001",
    "decision": "approve",
    "plan_status": "approved",
    "teacher_comment": "Plan validé."
  }
},

```

```
"errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "WF-08",
  "status": "forbidden",
  "error_code": "TEACHER_NOT_AUTHORIZED",
  "retryable": false,
  "data": {
    "learning_plan_id": 201
  },
  "errors": [
    {
      "field": "teacher_code",
      "reason": "teacher_mismatch"
    }
  ]
}
```

WF-09 - Régénération du plan

Rôle : Créer une nouvelle version du plan selon le commentaire enseignant.

Endpoint : POST /ai/learning-plan/regenerate

Tables utilisées : learning_plans, learning_plan_activities, workflow_events

Schéma du workflow

Webhook -> Load Previous Plan -> Load Activities -> Prepare Regeneration Context -> AI Agent -> Parse JSON
-> Insert New Plan Version -> Loop Activities -> Insert New Activities -> Call WF-07 -> Insert Event ->
Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir plan et commentaire.
2	Supabase Select learning_plans	Charger la version précédente.
3	Supabase Select activities	Charger les activités précédentes.
4	Function prepare_regeneration_context	Préparer le feedback.
5	Agent Plan Regeneration	Créer une nouvelle proposition.
6	Function parse_json	Parser la réponse.
7	Supabase Insert learning_plans	Insérer version +1.
8	Loop	Parcourir les nouvelles activités.
9	Supabase Insert learning_plan_activities	Insérer chaque activité.
10	HTTP Request WF-07	Valider la nouvelle version.
11	Supabase Insert workflow_events	Tracer la régénération.
12	Respond to Webhook	Retourner la nouvelle version.

Règles principales

- L'ancienne version est conservée.
- Le nouveau plan revient à pending_teacher_validation.

Input

```
{
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "learning_plan_id": 201,
  "teacher_code": "T001",
  "teacher_comment": "Réduire le plan à cinq jours et ajouter plus d'exercices.",
  "requested_duration_days": 5
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-09",
  "status": "plan_regenerated",
  "next_action": "teacher_plan_decision",
  "data": {
    "previous_learning_plan_id": 201,
    "new_learning_plan_id": 202,
    "plan_code": "PLAN-STD001-PYTHON",
    "previous_version": 1,
    "new_version": 2,
    "duration_days": 5,
    "activities_stored": 4,
    "validation_score": 100,
    "plan_status": "pending_teacher_validation"
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "WF-09",
  "status": "regeneration_failed",
  "error_code": "INVALID_REGENERATED_PLAN",
  "retryable": true,
  "data": {
    "previous_learning_plan_id": 201
  },
  "errors": [
    {
      "field": "activities",
      "reason": "missing_activities"
    }
  ]
}
```

WF-10 - Notification étudiant et enseignant

Rôle : Envoyer uniquement un plan approuvé et tracer les deux notifications.

Endpoint : POST /notifications/send-plan

Tables utilisées : students, assessments, learning_plans, learning_plan_activities, workflow_events

Schéma du workflow

Webhook -> Load Plan -> If Approved? -> Load Student -> Load Activities -> Format Student Message -> Gmail Send Student -> Insert Event -> Format Teacher Confirmation -> Gmail/Slack Send Teacher -> Insert Event -> Update Plan Sent -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir learning_plan_id.
2	Supabase Select learning_plans	Charger et vérifier le statut.
3	If-Else plan_approved	Bloquer avec 403 sinon.
4	Supabase Select students	Charger l'email étudiant.
5	Supabase Select activities	Charger les activités.
6	Function format_student_message	Construire le message.
7	Gmail Send	Envoyer à l'étudiant.
8	Supabase Insert workflow_events	Tracer STUDENT_NOTIFIED.
9	Function format_teacher_confirmation	Construire la confirmation.
10	Gmail ou Slack Send	Notifier l'enseignant.
11	Supabase Insert workflow_events	Tracer TEACHER_NOTIFIED.
12	Supabase Update learning_plans	Mettre status=sent et notification_status=sent.

Ordre	Nœud	Fonction
13	Respond to Webhook	Retourner les statuts.

Règles principales

- Seul status=approved autorise l'envoi.
- Les messages sont stockés en TEXT dans workflow_events.

Input

```
{
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "learning_plan_id": 201,
  "student_code": "STD001",
  "teacher_code": "T001",
  "channel": "email"
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-004",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-10",
  "status": "notifications_sent",
  "next_action": "track_student_progress",
  "data": {
    "learning_plan_id": 201,
    "student_recipient": "yassine@example.com",
    "student_notification_status": "sent",
    "teacher_recipient": "teacher@example.com",
    "teacher_notification_status": "sent",
    "plan_status": "sent"
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "WF-10",
  "status": "forbidden",
  "error_code": "PLAN_NOT_APPROVED",
  "retryable": false,
  "data": {
    "learning_plan_id": 201,
    "plan_status": "pending_teacher_validation"
  },
  "errors": [
    {
      "field": "plan_status",
      "reason": "approved_status_required"
    }
  ]
}
```

7. ORCH-04 - Suivi et dashboard

ORCH-04 - Suivi et dashboard

Rôle : Mettre à jour une activité puis rendre les données visibles dans le dashboard.

Endpoint : POST /orchestrator/progress

Tables utilisées : learning_plans, learning_plan_activities, workflow_events

Schéma du workflow

Webhook -> Initialize Progress Context -> WF-11 -> If Success? -> Build Response -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir la progression.
2	Function initialize_progress_context	Valider le statut et le pourcentage.
3	HTTP Request WF-11	Mettre à jour activité et plan.
4	If-Else success	Router erreur ou succès.
5	Function build_response	Construire le retour.
6	Respond to Webhook	Retourner la progression.

Règles principales

- Le dashboard est lu séparément via WF-12.

Input

```
{
  "transaction_id": "TX-005",
  "correlation_id": "CORR-001",
  "learning_plan_id": 201,
  "activity_id": 301,
  "activity_status": "completed",
  "completion_percentage": 100
}
```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-005",
  "correlation_id": "CORR-001",
  "workflow_id": "ORCH-04",
  "status": "progress_updated",
  "next_action": "continue_learning_plan",
  "data": {
    "learning_plan_id": 201,
    "activity_id": 301,
    "activity_status": "completed",
    "activities_total": 4,
    "activities_completed": 2,
    "completion_rate": 50,
    "plan_status": "in_progress"
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "ORCH-04",
  "status": "progress_update_failed",
  "error_code": "INVALID_ACTIVITY",
  "retryable": false,
  "data": {
```



```

    "learning_plan_id": 201
  },
  "errors": [
    {
      "field": "activity_id",
      "reason": "activity_not_found"
    }
  ]
}

```

WF-11 - Mise à jour de progression

Rôle : Mettre à jour une activité et recalculer la progression globale.

Endpoint : POST /progress/update

Tables utilisées : learning_plans, learning_plan_activities, workflow_events

Schéma du workflow

Webhook -> Validate Input -> Verify Plan -> Verify Activity -> Update Activity -> Count Activities -> Calculate Completion Rate -> Determine Plan Status -> Update Plan -> Insert Event -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir la progression.
2	Function validate_input	Vérifier status et pourcentage.
3	Supabase Select learning_plans	Vérifier le plan.
4	Supabase Select activity	Vérifier l'activité.
5	Supabase Update activity	Mettre à jour status et completion_percentage.
6	Supabase Aggregate activities	Compter total et completed.
7	Function calculate_completion_rate	completed / total * 100.
8	Function determine_plan_status	sent, in_progress ou completed.
9	Supabase Update learning_plans	Mettre completion_rate et status.
10	Supabase Insert workflow_events	Tracer ACTIVITY_UPDATED.
11	Respond to Webhook	Retourner le progrès.

Règles principales

- activity_status : not_started, in_progress, completed.
- completion_percentage entre 0 et 100.

Input

```

{
  "transaction_id": "TX-005",
  "correlation_id": "CORR-001",
  "learning_plan_id": 201,
  "activity_id": 301,
  "activity_status": "completed",
  "completion_percentage": 100
}

```

Output succès

```

{
  "success": true,
  "transaction_id": "TX-005",
  "correlation_id": "CORR-001",
  "workflow_id": "WF-11",
  "status": "activity_progress_updated",
  "next_action": "refresh_dashboard",
  "data": {
    "learning_plan_id": 201,
    "activity_id": 301,
    "activity_status": "completed",
    "completion_percentage": 100,
    "activities_total": 4,
    "activities_completed": 2,

```

```

    "plan_completion_rate": 50,
    "plan_status": "in_progress"
  },
  "errors": []
}

```

Output erreur type

```

{
  "success": false,
  "workflow_id": "WF-11",
  "status": "invalid_progress",
  "error_code": "INVALID_PERCENTAGE",
  "retryable": false,
  "data": {
    "activity_id": 301
  },
  "errors": [
    {
      "field": "completion_percentage",
      "reason": "value_out_of_range"
    }
  ]
}

```

WF-12 - Dashboard API

Rôle : Retourner les KPI et données de visualisation à partir des tables relationnelles.

Endpoint : GET /dashboard/mvp

Tables utilisées : students, assessments, student_results, student_skill_results, learning_plans, learning_plan_activities

Schéma du workflow

Webhook -> Normalize Filters -> Query Summary KPIs -> Query Priority Distribution -> Query Plan Statuses -> Query Skill Gaps -> Query Student Table -> Merge -> Format Response -> Respond

Nœuds Fusion AI

Ordre	Nœud	Fonction
1	Webhook Trigger	Recevoir les filtres.
2	Function normalize_filters	Nettoyer filtres vides.
3	Supabase Query summary	Compter étudiants, lacunes, plans.
4	Supabase Query priorities	Grouper par priority_level.
5	Supabase Query plan statuses	Grouper par status.
6	Supabase Query skill gaps	Grouper par skill_code et calculer moyenne.
7	Supabase Query students	Construire le tableau opérationnel.
8	Merge	Assembler les datasets.
9	Function format_dashboard_response	Structurer summary, distributions et students.
10	Respond to Webhook	Retourner le dashboard.

Règles principales

- Lecture uniquement.
- Les agrégations sont réalisées en SQL/Supabase plutôt que dans l'IA.

Input

```

{
  "class_code": "L1-INFO-A",
  "assessment_code": "ASSESS-PYTHON-001",
  "subject": "Python",
  "priority_level": "high",
  "plan_status": "pending_teacher_validation"
}

```

Output succès

```
{
  "success": true,
  "transaction_id": "TX-DASH-001",
  "workflow_id": "WF-12",
  "status": "dashboard_ready",
  "data": {
    "summary": {
      "students_evaluated": 25,
      "students_with_gaps": 18,
      "critical_students": 6,
      "plans_generated": 18,
      "plans_pending_validation": 4,
      "plans_approved": 12,
      "plans_sent": 10,
      "plans_completed": 2,
      "average_progress": 42
    },
    "priority_distribution": {
      "standard": 7,
      "low": 3,
      "medium": 9,
      "high": 6
    },
    "plan_status_distribution": {
      "generated": 2,
      "pending_teacher_validation": 4,
      "approved": 2,
      "sent": 5,
      "in_progress": 3,
      "completed": 2
    },
    "skill_gap_distribution": [
      {
        "skill_code": "PY-L00PS",
        "skill_name": "Boucles",
        "students_count": 12,
        "average_score": 38
      }
    ],
    "students": [
      {
        "student_code": "STD001",
        "student_name": "Yassine El Amrani",
        "global_score": 55,
        "main_gap": "Boucles",
        "priority_level": "high",
        "plan_status": "in_progress",
        "completion_rate": 50
      }
    ]
  },
  "errors": []
}
```

Output erreur type

```
{
  "success": false,
  "workflow_id": "WF-12",
  "status": "dashboard_failed",
  "error_code": "DASHBOARD_QUERY_FAILED",
  "retryable": true,
  "data": null,
  "errors": [
    {
      "field": "database",
      "reason": "query_failed"
    }
  ]
}
```

}
1

8. Tables Supabase minimales

Le MVP utilise sept tables relationnelles. Toutes les colonnes sont de type simple : TEXT, INTEGER, NUMERIC, BOOLEAN ou TIMESTAMP. Aucun JSON/JSONB n'est obligatoire pour les inserts depuis les nœuds Fusion AI.

students

Étudiants, classe et enseignant associé

Colonne	Type
id	bigint PK
student_code	text unique
full_name	text
email	text
class_code	text
teacher_code	text
created_at	timestampz
updated_at	timestampz

assessments

Évaluations et contexte pédagogique

Colonne	Type
id	bigint PK
assessment_code	text unique
title	text
subject	text
class_code	text
teacher_code	text
created_at	timestampz
updated_at	timestampz

student_results

Résultat global, priorité et diagnostic

Colonne	Type
id	bigint PK
student_id	bigint FK
assessment_id	bigint FK
global_score	numeric
priority_level	text
analysis_status	text
diagnosis_summary	text
diagnosis_strengths	text
diagnosis_weaknesses	text
pedagogical_impact	text
priority_justification	text
confidence	text
model_name	text
idempotency_key	text unique
created_at	timestampz
updated_at	timestampz

student_skill_results

Une ligne par compétence

Colonne	Type
id	bigint PK
student_result_id	bigint FK
skill_code	text
skill_name	text
score	numeric
max_score	numeric
percentage	numeric
gap_level	text

Colonne	Type
created_at	timestampz
updated_at	timestampz

learning_plans

Une ligne par version de plan

Colonne	Type
id	bigint PK
plan_code	text
student_result_id	bigint FK
version	integer
duration_days	integer
plan_title	text
objective_summary	text
success_criteria_summary	text
status	text
teacher_decision	text
teacher_comment	text
completion_rate	numeric
notification_status	text
created_at	timestampz
updated_at	timestampz

learning_plan_activities

Activités, ressources et progression

Colonne	Type
id	bigint PK
learning_plan_id	bigint FK
activity_order	integer
skill_code	text
resource_code	text
title	text
instruction	text
resource_url	text
resource_type	text
estimated_minutes	integer
target_score	numeric
activity_status	text
completion_percentage	numeric
completed_at	timestampz
created_at	timestampz
updated_at	timestampz

workflow_events

Logs, décisions et notifications

Colonne	Type
id	bigint PK
transaction_id	text
correlation_id	text
workflow_id	text
student_id	bigint nullable
assessment_id	bigint nullable
plan_id	bigint nullable
event_type	text
status	text
channel	text nullable
recipient	text nullable
message	text nullable
duration_ms	integer nullable
error_message	text nullable
created_at	timestampz

Relations principales

```

students 1 --- N student_results N --- 1 assessments
student_results 1 --- N student_skill_results
student_results 1 --- N learning_plans
learning_plans 1 --- N learning_plan_activities
workflow_events référence facultativement student_id, assessment_id ou plan_id

```

Règle d'insertion sans JSON/JSONB

```

skills[]
-> Function prepare_skill_rows
-> Loop
-> Supabase Insert student_skill_results (une ligne par compétence)

activities[]
-> Function prepare_activity_rows
-> Loop
-> Supabase Insert learning_plan_activities (une ligne par activité)

```

9. Dashboard MVP

Filtres

- Classe
- Évaluation
- Matière
- Priorité
- Statut du plan

Cartes KPI

KPI	Source
Étudiants évalués	COUNT DISTINCT student_results.student_id
Étudiants avec lacunes	student_results.analysis_status = gaps_detected
Cas critiques	student_results.priority_level = high
Plans générés	COUNT learning_plans
Plans en attente	learning_plans.status = pending_teacher_validation
Plans approuvés	teacher_decision = approve
Plans envoyés	status = sent
Progression moyenne	AVG learning_plans.completion_rate

Visualisations

Visualisation	Données
Barres - Lacunes par compétence	COUNT par skill_code et AVG percentage depuis student_skill_results
Répartition des priorités	COUNT par priority_level depuis student_results
Statut des plans	COUNT par status depuis learning_plans
Progression	completion_rate et activités terminées
Tableau enseignant	Étudiant, score, lacune principale, priorité, plan, progression, action

10. Ordre d'implémentation et tests

Priorité	Éléments
P0 - Base de données	Créer les 7 tables et contraintes uniques.
P1 - Analyse	WF-01, WF-02, WF-03 puis ORCH-01 sans IA.
P2 - IA diagnostic	WF-04 avec validation stricte.
P3 - Plan	WF-05, WF-06, WF-07 puis ORCH-02.
P4 - Humain et envoi	WF-08, WF-09, WF-10 puis ORCH-03.
P5 - Suivi et dashboard	WF-11, WF-12 puis ORCH-04 et interface.

Tests minimums

- Résultat valide de 3 compétences.
- Résultat invalide avec score hors limites.
- Replay du même idempotency_key sans doublon.
- Vérification skills_received = skills_stored.
- Classification correcte aux limites 34/35, 49/50 et 69/70.
- Diagnostic IA sans compétence inventée.
- Plan avec activité liée à une compétence non faible : rejet WF-07.
- Régénération créant version 2 sans écraser version 1.
- Blocage de WF-10 si plan non approuvé.
- Progression 100 % mettant le plan à completed.
- Dashboard avec et sans filtres.

Fin du document.